

الأساسيات - Eloquent ORM :الدرس 6

المحتويات

1. Eloquent ORM مقدمة عن
2. Models إنشاء
3. استرجاع البيانات
4. إنشاء السجلات
5. تحديث السجلات
6. حذف السجلات
7. Soft Deletes
8. Query Scopes
9. Accessors & Mutators
10. Mass Assignment
11. Timestamps
12. أمثلة عملية

Eloquent ORM مقدمة عن

ORM ما هو

ORM تعني **Object-Relational Mapping** (التعيين العلائقي للكائنات)

PHP Object (Model)
↓ يقوم بالربط بينهما
Database Table

Eloquent لماذا

SQL Injection حماية تلقائية من Objects بطريقة سهلة وواضحة ^١ التعامل مع البيانات كـ SQL كتابة استعلامات ^٢ علاقات قوية بين الجداول ^٣ توفير الوقت والجهد

المثال التقليدي vs Eloquent

طريقة SQL التقليدية:

```
$results = DB::select('SELECT * FROM users WHERE active = ?', [1]);
foreach ($results as $row) {
    echo $row->name;
}
```

طريقة Eloquent:

```
$users = User::where('active', 1)->get();
foreach ($users as $user) {
    echo $user->name;
}
```

إنشاء Models

بسيط Model إنشاء

```
php artisan make:model Product
```

يتم إنشاء الملف في: `app/Models/Product.php`

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    //
}
```

Model إنشاء مع Migration

```
php artisan make:model Product -m
```

إنشاء كل شيء مرة واحدة

```
php artisan make:model Product -mfsc
```

الخيارات:

- -m: Migration
- -f: Factory
- -s: Seeder
- -c: Controller

الأساسية Model خصائص

```
class Product extends Model
{
    // يحدده تلقائياً Laravel - اسم الجدول (اختياري)
    protected $table = 'products';

    // المفتاح الأساسي
    protected $primaryKey = 'id';

    // نوع المفتاح
    protected $keyType = 'int';

    // هل المفتاح يزيد تلقائياً؟
    public $incrementing = true;

    // هل نستخدم Timestamps؟
    public $timestamps = true;

    // Timestamps تخصيص أسماء
    const CREATED_AT = 'created_at';
    const UPDATED_AT = 'updated_at';
}
```

استرجاع البيانات

1. استرجاع جميع السجلات

```
// جميع المنتجات
$products = Product::all();

// جميع المنتجات مع أعمدة محددة
$products = Product::all(['name', 'price']);
```

2. البحث بالمفتاح الأساسي

```
// البحث عن منتج برقم 1
$product = Product::find(1);

// البحث عن عدة منتجات
$products = Product::find([1, 2, 3]);

// البحث أو رمي استثناء
$product = Product::findOrFail(1);
```

3. Where استعلامات

```
// شرط واحد
$products = Product::where('price', '>', 100)->get();

// شروط متعددة
$products = Product::where('category', 'Electronics')
    ->where('price', '<', 1000)
    ->get();

// OR condition
$products = Product::where('category', 'Electronics')
    ->orWhere('category', 'Computers')
    ->get();

// WHERE IN
$products = Product::whereIn('id', [1, 2, 3])->get();

// WHERE BETWEEN
$products = Product::whereBetween('price', [100, 500])->get();

// WHERE NULL
$products = Product::whereNull('deleted_at')->get();

// WHERE DATE
$products = Product::whereDate('created_at', '2024-01-01')->get();
```

4. الترتيب والتحديد

```
// الترتيب
$products = Product::orderBy('price', 'desc')->get();
$products = Product::orderBy('name')->get(); // تصاعدي افتراضياً

// أحدث وأقدم
$products = Product::latest()->get(); // حسب created_at
$products = Product::oldest()->get();

// تحديد عدد النتائج
$products = Product::take(10)->get();
$products = Product::limit(10)->get();

// التخطي والتحديد
$products = Product::skip(10)->take(5)->get();
```

5. استرجاع سجل واحد

```
// أول سجل
$product = Product::first();

// أول سجل بشرط
$product = Product::where('active', 1)->first();

// أول سجل أو رمي استثناء
$product = Product::where('slug', 'laptop')->firstOrFail();

// أول سجل أو إنشاء جديد
$product = Product::firstOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Laptop', 'price' => 5000]
);

// جديد (بدون حفظ Model أول سجل أو إرجاع)
$product = Product::firstOrCreate(['slug' => 'laptop']);
```

6. العد والتجميع

```
// عد السجلات
$count = Product::count();
$count = Product::where('active', 1)->count();

// أقصى وأدنى قيمة
```

```
$max = Product::max('price');
$min = Product::min('price');

// المجموع والمتوسط
$sum = Product::sum('price');
$avg = Product::avg('price');
```

7. Chunking (للبانات الكبيرة)

```
// معالجة البيانات على دفعات
Product::chunk(100, function ($products) {
    foreach ($products as $product) {
        // معالجة كل منتج
    }
});

// Lazy Loading
Product::lazy()->each(function ($product) {
    // معالجة كل منتج
});
```

إنشاء السجلات

1. طريقة create()

```
// إنشاء وحفظ مباشرة
$product = Product::create([
    'name' => 'Laptop',
    'slug' => 'laptop',
    'price' => 5000,
    'stock' => 10,
]);

// يجب تحديد الحقول المسموح بها في Model:
protected $fillable = ['name', 'slug', 'price', 'stock'];
```

2. طريقة save()

```
// إنشاء Instance جديد
$product = new Product;
```

```
$product->name = 'Laptop';
$product->slug = 'laptop';
$product->price = 5000;
$product->stock = 10;
$product->save();
```

3. إنشاء عدة سجلات

```
Product::insert([
    ['name' => 'Product 1', 'price' => 100],
    ['name' => 'Product 2', 'price' => 200],
    ['name' => 'Product 3', 'price' => 300],
]);

// ملاحظة: insert() لا يضيف $fillable ولا timestamps
```

4. firstOrCreate & updateOrCreate

```
// البحث أو الإنشاء
$product = Product::firstOrCreate(
    ['slug' => 'laptop'],          // شروط البحث
    ['name' => 'Laptop', 'price' => 5000] // البيانات للإنشاء
);

// التحديث أو الإنشاء
$product = Product::updateOrCreate(
    ['slug' => 'laptop'],
    ['name' => 'Gaming Laptop', 'price' => 7000]
);
```

تحديث السجلات

1. تحديث سجل واحد

```
// البحث ثم التحديث
$product = Product::find(1);
$product->price = 6000;
$product->save();

// تحديث مباشر
```

```
$product = Product::find(1);  
$product->update(['price' => 6000]);
```

2. تحديث عدة سجلات

```
// تحديث جميع المنتجات  
Product::where('category', 'Electronics')  
->update(['discount' => 10]);
```

3. Increment & Decrement

```
// زيادة  
$product = Product::find(1);  
$product->increment('views');           // +1  
$product->increment('views', 5);        // +5  
  
// نقصان  
$product->decrement('stock');           // -1  
$product->decrement('stock', 2);        // -2  
  
// مع تحديث حقول أخرى  
$product->increment('orders', 1, ['last_ordered_at' => now()]);
```

حذف السجلات

1. حذف سجل واحد

```
// البحث ثم الحذف  
$product = Product::find(1);  
$product->delete();  
  
// حذف مباشر  
Product::destroy(1);  
  
// حذف عدة سجلات  
Product::destroy([1, 2, 3]);  
Product::destroy(1, 2, 3);
```


2. حذف بشرط

```
// حذف جميع المنتجات غير النشطة
Product::where('active', 0)->delete();
```

3. حذف جميع السجلات

```
// حذف الكل (خطير)
Product::truncate();
```

Soft Deletes

ما هو Soft Delete؟

Soft Delete = `deleted_at` (الحذف الناعم) لا يحذف السجل فعلياً، بل يضع تاريخ في حقل

إعداد Soft Deletes

1. Migration في:

```
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->decimal('price', 10, 2);
    $table->timestamps();
    $table->softDeletes(); // يضيف عمود deleted_at
});
```

2. Model في:

```
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use SoftDeletes;
}
```

Soft Deletes استخدام

```
// حذف ناعم
$product = Product::find(1);
$product->delete(); // يضع التاريخ في deleted_at

// استرجاع السجلات المحذوفة
$products = Product::withTrashed()->get();

// فقط المحذوفة
$products = Product::onlyTrashed()->get();

// استعادة سجل محذوف
$product = Product::withTrashed()->find(1);
$product->restore();

// استعادة عدة سجلات
Product::onlyTrashed()
    ->where('category', 'Old')
    ->restore();

// حذف نهائي
$product = Product::withTrashed()->find(1);
$product->forceDelete();

// التحقق من الحذف
if ($product->trashed()) {
    echo "محذوف";
}
```

Query Scopes

Scope ما هو

Scope = Model استعمال قابل لإعادة الاستخدام في

Local Scopes

```
class Product extends Model
{
    // Scope للمنتجات النشطة
    public function scopeActive($query)
```

```

{
    return $query->where('active', 1);
}

// Scope للمنتجات الرخيصة
public function scopeCheap($query, $maxPrice = 100)
{
    return $query->where('price', '<=', $maxPrice);
}

// Scope للمنتجات الأكثر مبيعاً
public function scopePopular($query)
{
    return $query->where('sales', '>', 100)
        ->orderBy('sales', 'desc');
}
}

```

الاستخدام:

```

// منتجات نشطة
$products = Product::active()->get();

// منتجات نشطة ورخيصة
$products = Product::active()->cheap(50)->get();

// منتجات شائعة
$products = Product::popular()->take(10)->get();

// دمج Scopes
$products = Product::active()
    ->cheap(200)
    ->orderBy('name')
    ->get();

```

Global Scopes

```

// في Model
protected static function booted()
{
    static::addGlobalScope('active', function ($query) {
        $query->where('active', 1);
    });
}

```

```
// الآن جميع الاستعلامات ستشمل هذا الشرط تلقائياً
$products = Product::all(); // فقط النشطة

تجاهل // Global Scope
$products = Product::withoutGlobalScope('active')->get();
```

Accessors & Mutators

Accessors (القراءة)

Accessor = تعديل القيمة عند قراءتها

```
class Product extends Model
{
    // Accessor للاسم
    protected function name(): Attribute
    {
        return Attribute::make(
            get: fn ($value) => ucfirst($value), // حرف كبير أول
        );
    }

    // Accessor للسعر مع العملة
    protected function price(): Attribute
    {
        return Attribute::make(
            get: fn ($value) => number_format($value, 2) . ' ريال',
        );
    }

    // Accessor جديد (ليس في Database)
    protected function fullName(): Attribute
    {
        return Attribute::make(
            get: fn () => "{$this->name} - {$this->category}",
        );
    }
}
```

الاستخدام:

```
$product = Product::find(1);
echo $product->name;          // Laptop (حرف كبير تلقائياً)
echo $product->price;          // 5,000.00 ريال
echo $product->full_name;      // Laptop - Electronics
```

Mutators (الكتابة)

Mutator = تعديل القيمة عند حفظها

```
class Product extends Model
{
    // Mutator للاسم
    protected function name(): Attribute
    {
        return Attribute::make(
            set: fn ($value) => strtolower($value), // حفظ بأحرف صغيرة
        );
    }

    // Mutator لإزالة العملة (السعر)
    protected function price(): Attribute
    {
        return Attribute::make(
            set: fn ($value) => (float) str_replace(',', '', $value),
        );
    }
}
```

الاستخدام:

```
$product = new Product;
$product->name = 'LAPTOP'; // يُحفظ: laptop
$product->price = '5,000'; // يُحفظ: 5000
$product->save();
```

مماً Accessor & Mutator

```
protected function price(): Attribute
{
    return Attribute::make(
        get: fn ($value) => number_format($value, 2) . ' ريال',
        set: fn ($value) => (float) str_replace(',', '', $value),
    );
}
```

```
}
```

Mass Assignment

Mass Assignment ما هو

إنشاء/تحديث عدة حقول مرة واحدة

```
Product::create([
  'name' => 'Laptop',
  'price' => 5000,
  'stock' => 10,
]);
```

الحماية: fillable vs guarded

1. \$fillable (السماح):

```
class Product extends Model
{
  // الحقول المسموح بها فقط
  protected $fillable = [
    'name',
    'slug',
    'price',
    'stock',
  ];
}
```

2. \$guarded (المنع):

```
class Product extends Model
{
  // الحقول الممنوعة فقط
  protected $guarded = [
    'id',
    'created_at',
    'updated_at',
  ];
}
```

```
//أو السماح بكل الحقول (غير آمن)
protected $guarded = [];
}
```

لماذا الحماية مهمة؟

```
// بدون حماية، المستخدم يمكنه:
Product::create($request->all());

// إذا أرسل المستخدم:
// ['name' => 'Laptop', 'price' => 1, 'is_admin' => 1]
// أيضاً is_admin سيتم حفظ
```

Timestamps

الافتراضية Timestamps

```
class Product extends Model
{
    public $timestamps = true; // افتراضي

    // Laravel تلقائياً يضيف:
    // created_at
    // updated_at
}
```

تخصيص Timestamps

```
class Product extends Model
{
    // تغيير الأسماء
    const CREATED_AT = 'creation_date';
    const UPDATED_AT = 'last_update';

    // أو تعطيل Timestamps
    public $timestamps = false;
}
```

Timestamps التحكم في

```
// حفظ بدون تحديث updated_at
$product->saveQuietly();

// أو
$product->timestamps = false;
$product->save();

// يدوياً Timestamps تحديث
$product->touch();

// فقط updated_at تحديث
$product->update(['updated_at' => now()]);
```

أمثلة عملية

مثال 1: نظام المنتجات

```
// إنشاء منتج جديد
$product = Product::create([
    'name' => 'Gaming Laptop',
    'slug' => 'gaming-laptop',
    'description' => 'High performance laptop',
    'price' => 8000,
    'stock' => 5,
    'category' => 'Electronics',
]);

// عرض منتجات الإلكترونيات فقط
$electronics = Product::where('category', 'Electronics')
    ->active()
    ->orderBy('price', 'desc')
    ->get();

// تحديث المخزون بعد طلب
$product = Product::find(1);
$product->decrement('stock');
$product->increment('sales');

// حذف المنتجات القديمة
Product::where('stock', 0)
```



```
->where('updated_at', '<', now()->subMonths(6))
->delete();
```

مثال 2: نظام المدونة

```
// إنشاء مقال
$post = Post::create([
    'title' => 'Learn Laravel',
    'slug' => 'learn-laravel',
    'content' => 'Full content here...',
    'user_id' => auth()->id(),
    'published_at' => now(),
]);

// عرض المقالات المنشورة فقط
$posts = Post::where('published_at', '<=', now())
    ->orderBy('published_at', 'desc')
    ->paginate(10);

// زيادة المشاهدات
$post->increment('views');

// مقالات شائعة
$popular = Post::where('views', '>', 1000)
    ->orderBy('views', 'desc')
    ->take(5)
    ->get();
```

مثال 3: نظام المستخدمين

```
// إنشاء مستخدم
$user = User::create([
    'name' => 'Ahmed',
    'email' => 'ahmed@example.com',
    'password' => Hash::make('password'),
]);

// البحث بالبريد
$user = User::where('email', 'ahmed@example.com')->first();

// تحديث الملف الشخصي
$user->update([
    'name' => 'Ahmed Ali',
    'phone' => '0501234567',
]);
```

```
// تفعيل المستخدم
$user->email_verified_at = now();
$user->save();
```

نصائح مهمة

أفضل الممارسات ٩

1. الخام SQL بدلاً من Eloquent استخدم:

```
// سيء ٩
DB::select('SELECT * FROM products WHERE active = 1');
```

```
// جيد ٩ Product::where('active', 1)->get();
```

```
2. ** استخدم Query Scopes: **
```php
// تكرار الكود ٩
Product::where('active', 1)->where('price', '<', 100)->get();
Product::where('active', 1)->orderBy('name')->get();

// قابل لإعادة الاستخدام Scope ٩
Product::active()->cheap(100)->get();
Product::active()->orderBy('name')->get();
```

#### 3. للعرض فقط Accessors استخدم:

```
// صحيح ٩
protected function formattedPrice(): Attribute
{
 return Attribute::make(
 get: fn ($value) => number_format($this->price, 2),
);
}
```

#### 4. fillable/guarded احم البيانات بـ:

```
// آمن
protected $fillable = ['name', 'email'];
User::create($request->validated());
```

## ❗❗ أخطاء شائعة

### 1. نسيان fillable:

```
// خطأ: MassAssignmentException
Product::create(['name' => 'Test']);
```

// حدد fillable في Model protected \$fillable = ['name'];

```
2. **للبيانات الكبيرة all() استخدام**
```php  
// سيء: يستهلك الذاكرة  
$products = Product::all(); // سجل! 100,000  
  
// pagination أو chunk // جيد: استخدم  
Product::chunk(100, function ($products) {  
    // معالجة  
});
```

3. نسيان withTrashed():

```
// لن يجد السجل المحذوف  
$product = Product::find(1);
```

// صحيح \$product = Product::withTrashed()->find(1);

الخطوة التالية

بعد إتمام هذا الدرس، أنت الآن جاهز لـ:

7 العلاقات - Eloquent Relationships
- One to One

- One to Many
- Many to Many
- Polymorphic Relations

- - -

تعلم سعيد! 📖